



Option Chain Snapshot Analyser

For Nifty derivatives & equity options — quick PCR per strike, weekly max-pain detection.

PROJECT

ANALYTICS

Why this project matters



Key drivers

- Hands-on practice with market data ingestion & cleaning
- Bridges programming, data engineering, and financial logic
- Immediate visual outputs useful for trading research

📌 Audience: CS students & junior devs building financial analytics tools

Objectives & scope

Objective 1

Download option chain,
normalize fields

Objective 2

Compute Put-Call Ratio by strike
& expiry

Objective 3

Compute weekly Max-Pain &
visualize heatmap

Objective 4

Robust parsing for changing OI formats

Existing system vs proposed solution

Existing

- Manual CSV scraping or fragile scrapers
- One-off scripts, poor error handling
- No per-strike PCR or automated max-pain

Proposed

- Modular pipeline: fetch → normalize → compute → visualize
- Schema detection for OI/volume changes
- Automated reports (charts + downloadable snapshots)

Input data & project modules



Data Inputs

Exchange API / HTML scrape,
CSV dumps, ticker price feed

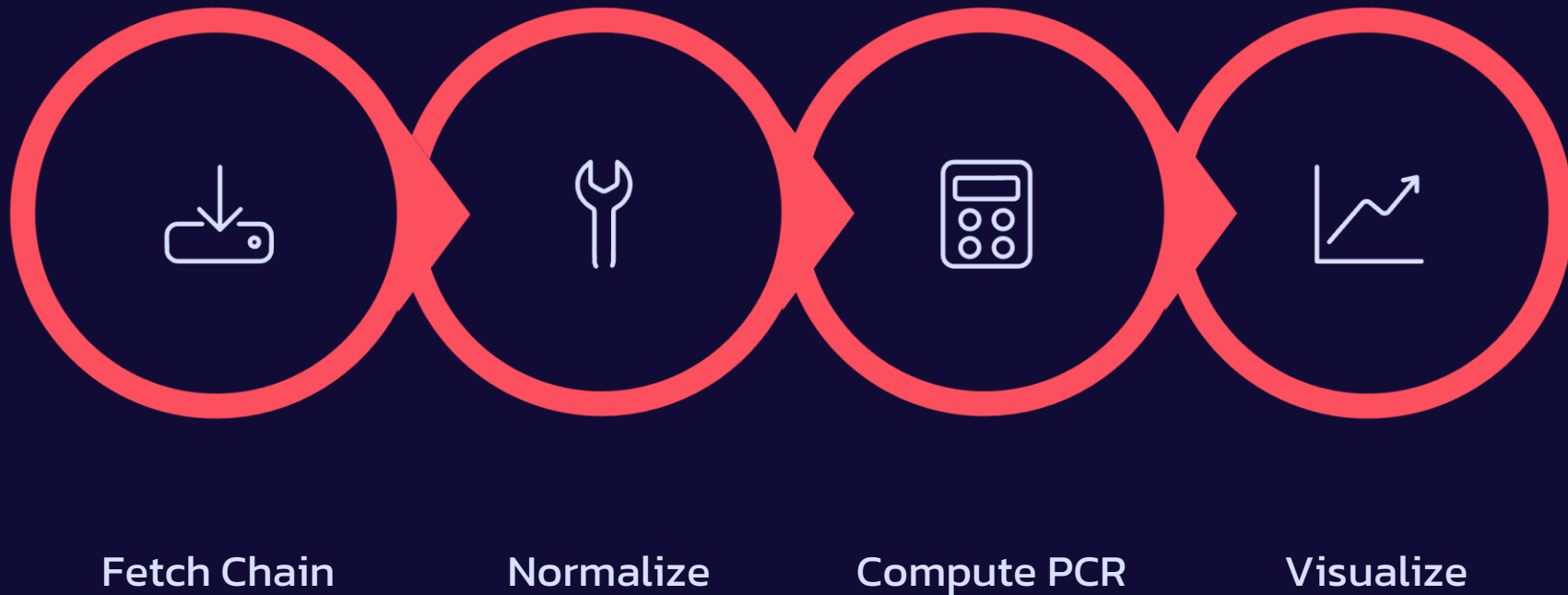
Modules

- Fetcher: API + resilient scraper
- Normalizer: parse OI, volumes, strike lists
- Calculator: PCR, max-pain engine
- Visualizer: charts, heatmaps, export

Outputs

Per-strike PCR table, weekly max-pain value, plots

System architecture & workflow



Pipeline is modular, testable, and re-runnable for each expiry.

$$\begin{aligned} X &= \left(\frac{R^2}{X_3} \frac{a^2}{0} + (\rho_3) x \right) = \left(\frac{x^2}{0} \frac{x}{2} \right) x(-2) + 1 \ell x + = \frac{x^2}{d_6} \frac{d^2}{a} \frac{v^2}{\ell^3} \\ x_3 x^2 &= \left(\frac{\lambda^2}{0_2} + x_3^2 + t(2) + (v^0) + (0) + a \right) N_0 \quad \left\{ u \right\} h = \left(\frac{x}{\ell} \right) \frac{H}{\ell^3} + + + (i x v)^{22} \\ L_3 x^{2^2} &= \frac{v^2}{a, 0} + t(x(v, a) + \frac{3}{0} +) X = a c^2 \quad \therefore, = h_3, \ell_3 x_3 + \frac{2 \ell i}{2 \ell i} \rightarrow \frac{0}{\ell} = \frac{1}{2} n^2 \\ X &= \left(\frac{2^2}{\ell n} \rightarrow (2) \text{ it } 0 \right) = \frac{F}{L} = g^2 = (h \times 3) \\ X &= \left(2 \rightarrow \left(\frac{2}{n} \right) = \frac{v^2}{0, a_3} \rightarrow a \frac{x \ell}{u_6} \frac{\ell x^2}{x \ell x_2} H \right)^2 \frac{a_3}{2} \frac{n^2}{\ell^3 - x_3} + x v = \ell_4^2 + (a_4 x) + (x d)^{22} \\ L \quad 2^2 \frac{x_2}{a^3} \frac{\ell}{10} &= \left(\frac{x^2}{\ell^2} + \frac{8 \ell}{\ell} \right) + - 1 \\ A = d) \frac{x \ell}{a x} + \frac{x_3}{(6, x, 22)} &\rightarrow (u x) = \left(\frac{u^2}{x_{j_3}} w \left(\frac{2 \times 2^3}{+ 2, 2} + a_3 \frac{x^3}{\ell} \right) \frac{1}{B 0_3} + \frac{\rho}{2} \right) = (N^2)^{x x} \\ v \quad \frac{x \ell^2}{\ell 0_8} &= \left(\frac{2 x^2}{2 \ell^3} + (x = (t, u, a_3) = (2)^2 = \frac{N^2 - x 0^2}{2 \ell_3 - x n} = N \frac{x}{j} - x_2 N = \left(\frac{2^2}{a_2} / x \right) + + (u_3^2)^{x^2} \right. \end{aligned}$$

Regex + fallback heuristics for OI formats
(commas, K/M, HTML shifts)

Special features & challenges



Data volatility

Strike range shifts when underlying gaps — dynamic strike windowing



Format drift

Auto-detect numeric formats; normalize K/M suffixes; ability to patch rules



Edge case: illiquid strikes with zero OI — exclude or flag for stability



Usability

Interactive heatmap, export CSV/PNG, weekly snapshot emails

Consider rate limits, retry backoff, and data caching to avoid API bans.

Class design & implementation notes

1

Fetcher

Methods:
fetch_current(),
fetch_historical()

2

Normalizer

Methods: parse_oi(),
normalize_strike_list()

3

Calculator

Methods: compute_pcr(),
compute_max_pain()

4

Visualizer

Methods:
plot_pcr_heatmap(),
export_snapshot()

OOP patterns

- Adapter pattern for multiple data sources
- Strategy pattern for parsing heuristics
- Factory for exporters (CSV, PNG, JSON)

Keep classes small, single responsibility, with unit tests for parsers and calculators.

Implementation, testing & next steps



Current progress

- Fetcher + basic normalizer implemented
- PCR & max-pain prototype working on sample data
- Visual heatmap prototype done

Next steps

1. Robustify parsers; add schema inference tests
2. Integrate rate-limit safe scheduler and caching
3. Polish UI, add export/email snapshots, write end-to-end tests

📌 Deliverable: reproducible pipeline + documentation + unit/integration tests